Purpose and authority

BFMR MVP 1 Solution Design Specification

Version 1.5 | 9 October 2026 | Prepared for Matt Ward

Receive a single new-deal email, identify the BFMR deal in an authenticated browser, check My Tracker for an existing active reservation, and reserve quantity 1 through the BFMR website. Confirm the result through My Tracker. Matt releases reservations manually.

Current scope

All retailers and bundle deals are permitted. MVP 1 performs no financial, profitability, retailer-stock or retailer-eligibility evaluation and makes no purchases. Multiple-deal emails and additional-slot announcements are ignored. Testing and production both make real reservations.

Authority and status

This specification replaces v1.4 for MVP 1 and incorporates the final decisions in the accompanying inconsistency-review conversation. Later explicit user answers override earlier answers, including intermediate choices withdrawn during that conversation. Earlier source documents are historical references where they conflict with this version.

Confirmed requirements are identified as such. Retained engineering decisions and new engineering consequences are distinguished from user requirements. This is an implementation design, not evidence that the system has been implemented or validated live.

Reading guide

Pages	Contents
2-4	Change register and requirements
5-10	Architecture, intake, browser/API contracts, state and recovery
11-15	Notifications, persistence, concurrency, configuration and operations
16-18	Acceptance tests and implementation gates
19-21	Deferred decisions, source defects and traceability

Resolved decisions and precedence

Topic	Authoritative v1.5 disposition
Reservation flow	Reserve quantity 1 after trigger/identity, duplicate and operational safety checks. No business-eligibility evaluation before or after reservation.
Retailers and bundles	Any retailer; bundles included. Amazon-only restrictions and bundle rejection are superseded.
Release and purchase	All releases are manual. No one-minute release, failed-eligibility release, cancellation API calls or retailer purchase in MVP 1.
Duplicate safeguard	Any existing active reservation for the same deal blocks a new reservation, including a manual reservation and any color.
Email scope	Single new-deal emails only. Ignore multiple-deal emails and additional-slot notifications. A bundle offer is one deal, even if it contains several products.
Subject syntax	Accept both historical NEW DEAL | retailer subjects and variants such as DEAL DAYS - NEW DEAL. Retailer channel is not required.
Downtime	Catch up and process qualifying emails received while stopped. Preserve the durable Gmail cursor; remove restart receipt-time exclusions.
Platform	Single Ubuntu host with Playwright bundled Chromium in headed mode; secured visible session for manual login/MFA.
Deal navigation	Open the validated BFMR email link directly in the dedicated authenticated browser. No mandatory Active Deals lookup first.
External operations	Playwright website form is the only reservation-creation route. My Tracker API performs duplicate and confirmation reads.
Testing	Both testing and production create real reservations. No default dry-run requirement; offline tests still use fake adapters.
Financial discussion	All financial decisions are excluded from MVP 1. Page 19 preserves them as deferred discussion, not instructions to implement.

Safety checks are not business eligibility

Email provenance, correct deal/account identity, duplicate prevention, quantity selection, reliable storage and uncertain-result reconciliation remain necessary to perform the authorized action correctly. They do not filter on retailer, price, payout, stock or bundle status.

Functional requirements

Stable FR IDs from v1.4 are retained with revised meanings where the user changed scope. FR-012 explicitly records manual release and the absence of purchasing.

ID	Confirmed required behavior
FR-001	Monitor one Gmail Inbox. Direct sender: deals@buyformeretail.com. Testing additionally permits forwards from matt.ward@live.com. Both modes create real reservations.
FR-002	Process single new-deal emails. Ignore multiple distinct deals, additional-slot announcements and unrelated mail. Do not require a retailer channel in the subject.
FR-003	Reserve regardless of retailer and include bundles. Do not evaluate financial criteria, Amazon stock, retailer permission or purchase eligibility. No maximum response-time guarantee.
FR-004	Read My Tracker for the configured BFMR account. Any matching deal_id in reserved status blocks creation, including manual reservations and all variants. A later different email can qualify after release.
FR-005	Submit quantity 1 total for one deal offer, not one unit per color. Any available color is acceptable. Preserve selected variant/color or bundle-offer identity for reconciliation.
FR-006	One active deal worker. Process oldest known qualifying email first. Failed or incomplete duplicate-status reads hold the queue until a complete read succeeds.
FR-007	Catch up after downtime and retain admitted unfinished work across crashes. Deduplicate delivery and processing; do not exclude mail merely because receipt preceded the current process start.
FR-008	Reconcile uncertain submissions, including after restart. Unresolved outcomes create review and Telegram alert; only the affected deal is blocked once uncertainty is classified.
FR-009	Confirmed non-creation permits one automatic retry. Second failure stops that deal and alerts. Closed/no-slot results are skipped without retry or alert.
FR-010	Local terminal commands resolve reviews and explicitly request reprocessing. Resolution does not submit. Reprocessing grants one initial attempt and one retry after safety checks.
FR-011	Duplicate delivery never automatically replays a completed email. New mail cannot bypass current-run review/exhaustion blocks. Explicit replay is limited to mail admitted in the current run.
FR-012	MVP 1 does not purchase or cancel. Leave successful reservations in place for Matt to manage and release manually. No timed release.

Operations and integration requirements

ID and status	Required behavior
OPS-001 | retained	Start on Ubuntu boot and recover automatically after crashes. Run-scoped admission blocks expire on restart; unresolved external attempts remain and are reconciled.
OPS-002 | retained	Maintain local activity logs. Login/MFA pauses new reservations, alerts and resumes after verified restoration. Temporary disconnection does not create a new run.
OPS-003 | retained	Use one private Telegram chat. One incident alert and one recovery per episode; no ordinary success alerts. Notification delivery failure halts new deal work until actual sends are accepted; intake continues.
CON-001 | revised	Gmail watch and Pub/Sub-driven intake. No periodic Gmail polling or Inbox sweep. Continue from a durable history cursor on startup/reconnect to catch up. Browser UI is the only reservation write route.
INT-001 | confirmed	Use My Tracker read API for existing-reservation checks and post-submit confirmation. No reservation-creation API is assumed or implemented.
INT-002 | confirmed	Navigate the email BFMR deal link in the dedicated authenticated Chromium profile. Do not require Active Deals/Deal Details mapping or retailer eligibility reads.
SEC-001 | retained design	Use a dedicated non-root service identity, restricted secrets/profile storage and one expected BFMR account. Stop on wrong-account, login or access challenges; no bypass.
SEC-002 | retained design	Treat email/page content as untrusted data. Restrict accepted deal URLs and redirects. Never execute instructions embedded in a deal or email.
DATA-001 | retained design	Use local PostgreSQL for durable intake, Gmail cursor, dedupe, commands, attempts, incidents and audit records. Commit notifications before ACK.
DATA-002 | retained design	Preserve attempt evidence and dedupe across restarts. Store external identifiers as text and timestamps in UTC. Do not persist secrets or raw account pages in business tables.

Boundaries and limitations

No dashboard, Telegram command interface, manual pause/resume command, checkout, shipment tracking or payout processing is added. The catch-up requirement applies while Gmail history remains available. Recovery from an expired history cursor remains an explicit contract gap under the retained no-sweep constraint; it must not be represented as guaranteed lossless recovery for arbitrarily long downtime.

Architecture and decisions

Component	Responsibility
Gmail watch and StreamingPull	Receive outbound Pub/Sub notifications; renew watch. No public webhook or tunnel.
Durable intake and history reader	Commit notifications before ACK; serialize cursor advancement; catch up after restart and reconnect; classify and admit email.
Single deal worker and browser owner	Resolve trusted BFMR link, identify deal/offer, check operational gates and submit through the website.
My Tracker API adapter	Complete account reservation reads and post-submit reconciliation, with server-side API credentials.
PostgreSQL	Persist queue, cursor, attempts, dedupe, incidents, notification outbox and audit events.
Notification dispatcher and local CLI	Deliver Telegram incidents independently; accept local status, review and reprocessing commands.

ADR-001 and ADR-002 | Retained

Single Node.js/TypeScript application on Ubuntu with local PostgreSQL. One account and serialized browser operations do not justify distributed services or a separate broker. SQLite remains viable, but PostgreSQL preserves the existing persistence direction and provides transactions and ownership locks. Cost: local database administration and backups.

ADR-003 | Revised and user approved

Direct authenticated navigation with UI submission and My Tracker reads. Alternative: paginate Active Deals to resolve the email code before navigation. Direct navigation removes that mandatory lookup and reaches the form that must be used anyway. The historical failure of unauthenticated/fresh-browser resolution does not prove authenticated navigation fails. Trade-off: stable deal identity must be extracted and verified from the browser page before tracker matching; page changes can require adapter maintenance. Speed advantage is expected, not benchmarked.

ADR-004 and ADR-005 | Persistence and catch-up

Durability does not authorize replay of completed work. Retain event evidence, per-command attempt budgets and run-scoped blocks separately. Revised design: persist the Gmail cursor across starts and perform one startup continuation after activating the listener. First-ever initialization and expired-cursor handling are explicit engineering boundaries on page 6, not silent restart cutoffs.

Gmail intake and downtime catch-up

Startup sequence

1. Validate configuration and Gmail identity; open PostgreSQL and obtain singleton ownership. Create a STARTING run. Preserve the existing consumed history cursor. Keep submissions behind the recovery barrier while starting independent intake.

2. Establish or renew the INBOX watch; record watch expiration and watch baseline separately from the consumption cursor. Start StreamingPull and atomically activate the run. Buffer callbacks until activation is committed. listener_started_at is an audit timestamp, not an email eligibility cutoff.

3. When a saved cursor exists, perform one startup history continuation from that cursor without waiting for a new notification. Process changes from downtime, then continue event-driven reads. Renewing the watch must never overwrite the saved consumption cursor.

First-ever initialization | Engineering decision

With no prior cursor and no restored history state, save the initial watch baseline as the initial consumption cursor, then continue from it. This begins monitoring from first setup; it does not scan historical Inbox contents. An existing installation missing its cursor is a recovery fault, not permission to silently initialize afresh. Historic mail before first setup is not authorized by the downtime catch-up decision.

Durable processing

Validate notification mailbox/shape. Insert under unique subscription/message ID, then ACK after commit. Verify existing duplicate records before ACK. Persist sanitized rejected-event evidence before acknowledging poison input. Do not ACK while persistence is unavailable.

Read Gmail history serially and consume all pages. Follow message-added and relevant Inbox-label-added events; fetch each distinct message and require INBOX in its fetched labels. Retain the prior v1.4 behavior for Inbox membership. Sorting is internalDate, then Gmail ID. Atomically persist classifications/admissions and advance the fully consumed checkpoint. Any required fetch/page failure leaves the cursor unchanged; database dedupe makes repetition harmless.

Reconnect, watch maintenance and history gaps

Temporary interruptions retain the run and cursor. Perform a one-time reconnect continuation, then resume event-driven reads; no quiet-mailbox polling. Renew watch daily and retain expiration separately. Delayed notifications may reveal earlier missed changes; no-poll operation cannot promise prompt discovery.

On an expired/rejected history cursor, raise GMAIL_HISTORY_GAP, retain the cursor and stop new intake. Restart must not erase the gap or silently reset the baseline. Already-admitted work may proceed if other gates permit. A full Inbox recovery sweep remains outside this design; guaranteed recovery beyond history retention requires a further explicit decision. Validate actual Gmail retention behavior before release.

Classification and deal identity

Sender and message classification

Production accepts direct mail from deals@buyformeretail.com. Testing also accepts forwards whose outer From mailbox is matt.ward@live.com. Normalize mailbox casing, not display names. Embedded forwarded headers are descriptive content, not proof of sender identity. Use trusted Gmail authentication evidence with fixture-backed rules; unknown authentication for a recognized candidate produces no reservation and a diagnostic alert.

Handle repeated Re/Fw/Fwd prefixes. Accept historical NEW DEAL | retailer and DEAL DAYS - NEW DEAL layouts without a required retailer channel. Preserve HTML hyperlink targets when plain text omits them. Count distinct deal identities, collapsing repeated links to the same deal; navigation/banner links do not count. Unambiguous multi-deal mail and additional-slot announcements are silent logged skips. Ambiguous single-deal identity creates review.

Direct BFMR navigation

Accept only HTTPS deal links on an exact approved BFMR hostname/path (initial known hosts: bfmr.com and www.bfmr.com; /deals/ path). Parse URLs, never use substring host matching. Reject embedded user credentials, unexpected ports and arbitrary destinations. Restrict top-level redirects to validated BFMR destinations; any necessary external login destination requires a separately verified authentication contract before use.

Navigate the email link in the dedicated authenticated browser. Verify expected account and establish stable BFMR deal_id, canonical page URL and the correct reservation form. Keep email deal code, canonical slug and deal_id distinct; never substitute the short URL token for an API slug or tracker ID. If the mapping cannot be established reliably, stop that deal for review. Active Deals API mapping is not a required step or automatic fallback.

Offer and quantity selection

Retailer, retail stock, price and payout do not gate MVP 1. Bundles are allowed. Interpret quantity 1 as one BFMR offer/reservation unit, which may contain multiple bundle components; do not reserve each component separately. This is an engineering interpretation of the confirmed bundle inclusion and quantity rule and must match the real form contract.

For alternative colors/variants choose one available reservable offer using stable variant ID order, or normalized label and stable form position if IDs are absent. Persist selected offer identity, color/description where applicable, and bundle-component identity when necessary to distinguish offers. Lack of retailer stock is not a reason to reject a deal. BFMR closed/no-slot availability is an execution constraint and produces a silent skip.

Browser and My Tracker contracts

Operation	Required result and behavior
verifySession(account)	AUTHENTICATED with expected identity; AUTH_REQUIRED, CHALLENGE or UNKNOWN otherwise. Wrong account blocks all submissions.
readDeal(candidate)	IDENTIFIED: stable deal_id, canonical page, reservation form and offered variants/bundle identity. No financial/retailer/stock eligibility result. Alternatives: UNAVAILABLE, AUTH_REQUIRED or REVIEW.
inspectReservations(account, deal)	GET My Tracker with API-KEY and API-SECRET. Match deal_id across all variants. Any reserved row means RESERVED. Return CONFIRMED_NONE only after complete, fresh, validated pagination; errors/ambiguity mean UNKNOWN.
submitOne(attempt, offer)	Normal website-form action, quantity 1 for exactly one offer. Return acknowledgement evidence or UNCERTAIN. Never internally repeat the click or request after submission may have begun.
reconcile(attempt)	CONFIRMED_ONE, CONFIRMED_NOT_CREATED, CONFLICT or UNKNOWN. Verify deal, quantity, offer/variant and RID where present. AUTH_REQUIRED causes global pause.

Tracker read contract and evidence

Use the supplied GET /api/v2/my-tracker endpoint with explicit date bounds covering at least the preceding 24 hours plus a configurable boundary buffer; consume every page. The 24-hour minimum derives from reported purge behavior, not the published response schema. Verify timezone, date-field meaning, pagination, status and RID semantics before live use. If older active reservations can exist, widen/remove the date filter so absence is never inferred from an incomplete window.

The published example contains empty objects and does not establish a complete response schema. Preserve raw field-name mappings only in sanitized fixtures. Check that API credentials and browser session refer to the same BFMR account. Separate transport, authentication, schema and incomplete-page failures from empty results.

Proof of creation and non-creation

A success toast is acknowledgement only. Final confirmation requires My Tracker to show the expected deal, quantity 1 and matching offer identity. Wrong quantity, mismatched offer or possible simultaneous manual creation is CONFLICT; do not cancel or modify it.

CONFIRMED_NOT_CREATED requires a definitive rejection or a complete, fresh tracker read under a validated immediate-visibility contract. The reported immediate appearance and manual-release removal require live evidence. If those guarantees cannot be established, absence after an uncertain click remains UNKNOWN and cannot authorize a retry. Read-only API keys stay outside the browser.

Command states and retry limits

State	Transition and effect
QUEUED	Claim oldest runnable command, check run/deal blocks, then CHECKING. One account worker.
CHECKING	Verify session and deal/offer identity, then complete duplicate check. Qualifying action -> READY; existing reservation/closed/no-slot -> SKIPPED; ambiguous identity -> REVIEW.
WAIT_STATUS	Failed or incomplete reservation-status read holds the queue. Retry reads, alert once and recover once; no submit attempt consumed.
READY	Recheck global gates and fresh absence. Commit next attempt as SUBMITTING before invoking submitOne.
SUBMITTING	One external invocation for this ordinal. Record available outcome, then RECONCILING even after apparent success.
RECONCILING	Target reads near 0, 10 and 30 seconds. Confirmed one -> RESERVED; proved non-creation -> RETRY_PENDING or EXHAUSTED; unresolved/conflict -> REVIEW.
RETRY_PENDING	After confirmed first-attempt non-creation only. Refresh identity, availability, gates and duplicate status before ordinal 2.
RESERVED / SKIPPED	Terminal. No automatic replay on duplicate delivery; no reservation-success notification or automatic release.
REVIEW / EXHAUSTED	Terminal automatic outcome. Current-run deal block and incident. Unrelated deals proceed once global blockers are clear.

Attempt budget

Maximum two submission ordinals per command: initial plus one retry. Read retries do not consume this budget. A possibly issued action does. A crash between committing SUBMITTING and the click is still uncertain and requires reconciliation. Restart never resets an unfinished command budget. No hidden submit retry inside generic browser or network helpers.

Closed/no-slots proved before or by a definitive submit response becomes SKIPPED_UNAVAILABLE with no retry/alert. A timer never turns an unknown outcome into non-creation. An explicit eligible CLI replay creates a new command and budget only after review resolution and live checks.

Scheduling

Order by original Gmail internalDate, Gmail ID and monotonic command sequence. Late-discovered downtime mail cannot undo a completed reservation. Finish or classify the active command before moving on. Manual replay retains its email priority without interrupting work. Application serialization cannot prevent a manual BFMR action between check and click; classify observed conflicts conservatively.

Restart recovery and operator controls

Persisted situation	Restart behavior
QUEUED / CHECKING / READY / WAIT_STATUS	Keep admitted work; restart checks using live facts. Preserve original admission and email receipt time; execute in the new run.
SUBMITTING / RECONCILING	Reconcile before any new submission. Confirmed matching creation -> RESERVED; proved non-creation -> remaining retry only; uncertainty/conflict -> current-run review block.
RETRY_PENDING	Retain attempt count and perform ordinal 2 only after fresh checks.
RESERVED / SKIPPED / REVIEW / EXHAUSTED	Do not automatically re-enqueue terminal commands. Expire old run-scoped admission blocks, but recreate blocks for unresolved external effects.
Email received during downtime	Continue the saved Gmail history cursor and admit qualifying unprocessed mail, regardless of listener start time. Apply ordinary dedupe and deal blocks.
Temporary outage / mode change	Outage retains run. Real restart may change mode; re-evaluate sender permission on unfinished backlog. Testing forwards become SKIPPED_MODE in production.

Recovery barrier

No new submission until every potentially issued unresolved attempt is safely resolved or isolated under a current-run review block. If account-status inspection itself fails, hold the queue. Intake and essential recording may continue. A restart is never a substitute for incident resolution. Telegram failure and Gmail history-gap state persist independently of run-scoped deal blocks.

Local CLI contract | Retained design

Command	Semantics
bfmr status / bfmr activity	Read mode, cursor/catch-up state, queued/carried work, active command, gates and sanitized history.
bfmr reviews list	List current-run review/exhaustion incidents; historical records remain read-only.
bfmr review resolve <id>	Require outcome, evidence/note and expected version. Outcomes: confirmed-created, confirmed-not-created, acknowledged-failure for known non-creation. Enqueue nothing.
bfmr emails list	Show admitted mail, retained backlog, terminal state and replay eligibility.
bfmr email reprocess <gmail-id>	Require current-run admission, no unresolved review and no active command. Create one idempotent CLI command with fresh two-attempt budget.

Downtime mail may enter the current admission run. Completed prior-run mail cannot be replayed by CLI. No cancellation or pause/resume command.

Telegram incidents and gates

Incident policy

Send one alert when a new incident opens and one recovery when the condition is proved resolved. Incidents include Gmail authorization/watch/history gaps, BFMR login or wrong account, failed reservation-status read, unidentifiable single deal, uncertain/conflicting submission and exhausted retry. Ordinary skips and successful reservations do not send alerts. Known multi-deal, extra-slot, existing-reservation and closed/no-slot outcomes are logged silently.

Acceptance controls recovery

Use sendMessage to the fixed private chat. Delivery acceptance requires a valid response with ok=true and a returned Message. HTTP reachability, DNS, getMe or a bot identity probe is not enough. Do not send a proactive startup test. Persist a known delivery failure before continuing deal work; retry the failed outbox item with backoff and Retry-After.

After accepted delivery, send the Telegram recovery message and drain required pending operational messages. Clear TELEGRAM_BLOCKED only after those sends are accepted. Intake, durable recording and essential reconciliation remain active while new identity checks/submissions are halted. If a click already began, finish recording/reconciling it; never interpret the new gate as proof nothing happened.

Gate	Clears when
PERSISTENCE_UNHEALTHY / OWNERSHIP_LOST	Storage and exclusive ownership are healthy and all possible in-flight effects reconciled.
STARTUP_RECONCILIATION	Every possible prior submission is resolved or isolated as review.
BFMR_AUTH_REQUIRED	Expected BFMR account identity/session verified after manual login/MFA.
RESERVATION_STATUS_UNAVAILABLE	Complete fresh status check succeeds for the blocked command.
TELEGRAM_BLOCKED	Pending failed notification, required recovery and pending operational messages are accepted.

Failure semantics

All active submission gates must clear before new work proceeds. Gmail intake loss alone does not invalidate already-durable work; failure to deliver its alert may independently halt it through TELEGRAM_BLOCKED. This retains the v1.4 operational design.

Accepted sends with lost responses may be retried and duplicated. Include a stable incident ID; never claim exactly-once Telegram delivery. Alert/recovery phases have unique outbox keys. Review/exhaustion incidents are resolved by a supported operator disposition, not timeout. Notification acceptance is not proof that Matt read the message.

Data model and invariants

Internal UUIDs; external IDs as text; UTC timestamps; JSONB only for bounded sanitized evidence. All commands retain email and account references. Browser/API secrets are never stored in business records.

Table	Core fields and invariants
runs	id, mode, created_at, listener_started_at, ended_at, config_version, status. Mode immutable while active. Start time is audit only, not admission cutoff.
inbound_events	id, subscription, provider_message_id, mailbox, history_id, run_id, state. UNIQUE(subscription, provider_message_id).
gmail_state	mailbox PK, consumed_history_id, watch_baseline_history_id, watch_expiration, last_success_at, intake_state, initialization marker. Cursor persists across runs; one writer.
emails	id, mailbox, gmail_id, internal_date, outer_sender, auth_result, inbox_at_fetch, admission_run_id, classification, candidate_refs, content_hash. UNIQUE(mailbox, gmail_id).
deal_observations	id, account_id, deal_id, email reference/code, canonical URL/slug where known, offer/variant identity, color, optional bundle components, observed_at. No eligibility evaluation fields required.
commands	id, email_id, admission_run_id, execution_run_id, AUTO/CLI kind, request_id UNIQUE, state, attempt_count 0..2, queue_time, monotonic sequence, deal_id.
attempts	id, command_id, ordinal 1..2, account/deal, selected offer identity, quantity CHECK=1, prepared_at, invoked_at nullable, result, reservation_id, evidence. UNIQUE(command_id, ordinal).
deal_blocks	id, run_id, account_id, deal_id, REVIEW/EXHAUSTED type, reason, resolved_at. Unique active block per run/account/deal/type.
incidents	id, scope, type, episode, ACTIVE/RESOLVED state, opened_at, resolved_at, disposition. One active incident per scope/type.
notification_outbox	id, incident_id, ALERT/RECOVERY phase, state, due_at, tries, telegram_message_id. UNIQUE(incident_id, phase).
runtime_gates / workflow_events	Gate name, reason, since; append-only events with actor, correlation/causation IDs, run/command/attempt IDs and sanitized detail.

Indexes and boundaries

One AUTO command per email; one active command per email across AUTO/CLI. Index queue ordering, unresolved attempts, active deal blocks and due notifications. Do not enforce permanent one-reservation-per-deal: manual release followed by a different email is valid. Review evidence survives expiry of run-scoped blocks. Financial, order, shipment and payout tables are outside MVP 1.

Transactions and ownership

Boundary	Atomic commit and crash rule
T1 Receipt	Persist validated/rejected notification, then ACK. Redelivery finds the unique key.
T2 History batch	Persist classifications, email dedupe and command admissions with the fully consumed cursor. Failed fetch/page leaves checkpoint unchanged.
T3 Attempt preparation	Lock command; check budget/gates; allocate ordinal; commit SUBMITTING with chosen offer and pre-submit evidence. No external click inside the transaction.
T4 Outcome	Record evidence, command/attempt result, deal block, incident, notification and audit event atomically.
T5 Operator action	Validate request ID and expected entity version; record resolution or enqueue one CLI command with audit.
T6 Notification	Record accepted Telegram ID/phase and clear gate only after required accepted sends. Lost response leaves uncertain delivery.

Exclusive browser ownership

Use a process singleton lock plus a dedicated PostgreSQL advisory lock for the BFMR account. A second instance cannot start a listener run or submit. One browser owner serializes all page operations; health/auth probes and the CLI cannot race the worker. The CLI submits requests to the worker and never drives the browser or mutates tables directly.

systemd must terminate the old process group, including Chromium, before replacement. Reacquiring a database lock alone is not proof an old browser stopped. On connection/lock-health loss, cease initiating browser actions immediately. The supervisor terminates an unresponsive owner; recovery reconciles possible side effects. Multiple hosts sharing the same BFMR account are unsupported.

External uncertainty

A local commit and BFMR form submission are not atomic. A crash after T3 may occur before or after the click. Reconcile both cases. The design gives one application submit invocation per recorded attempt while ownership is healthy, not a mathematical exactly-once guarantee from BFMR. Three reads are a bounded investigation, not permission to retry after 30 seconds.

Backpressure

Bound Pub/Sub in-flight delivery and Gmail fetch concurrency. ACK only durable notifications. Storage pressure delays ACK safely and may cause duplicate delivery. Raise an incident and stop new effects before storage becomes unusable. Keep the authoritative queue in PostgreSQL, never solely in memory. Audit failures must not cause an unrecorded blind retry.

Configuration and local interfaces

Configuration	Contract
Mode and identities	MODE=testing|production is mandatory and immutable until restart. GMAIL_ACCOUNT_EMAIL and BFMR_EXPECTED_ACCOUNT are required. Validate Gmail OAuth and browser/API BFMR account alignment.
Gmail and Pub/Sub	Configured project/topic/subscription and credential references. Gmail refresh credentials, watch expiration and persistent cursor are distinct concerns.
BFMR API	BFMR_API_KEY and BFMR_API_SECRET references are required for My Tracker reads. Keep server-side. Reservation UI does not eliminate tracker authentication requirements.
Browser	Pinned Playwright version and matching bundled Chromium; dedicated persistent profile path; headed mode and secured desktop session. No Edge dependency.
Database and logs	Local database connection secret reference, restricted log directory, backup destination and retention. Configuration/schema version recorded with runs.
Telegram	One fixed private chat ID and secret bot-token reference. No incoming Telegram command endpoint.
Read controls	Tracker date-window/buffer settings, validated pagination contract, finite API/browser timeouts and retry policy. No business eligibility thresholds used in MVP 1.

Defaults | Engineering decisions

Renew Gmail watch daily. Read/notification retries begin at 5 seconds and back off exponentially to 60 seconds with jitter; honor Retry-After. Reconciliation targets 0/10/30-second reads but must respect validated provider consistency. Unknown submit outcomes never flow through generic retries. Keep ordinary logs/intake evidence for 180 days; preserve unresolved effects and minimal dedupe longer.

CLI transport

Use a permission-restricted Unix socket under /run/bfmr. Each request carries schema version and request_id; repeating the same request ID returns the same result. Reject unknown fields. Resolve/reprocess operations require expected version checks. Stable errors: NOT_CURRENT_RUN, REVIEW_UNRESOLVED, ALREADY_ACTIVE, UNKNOWN_ID, NOT_AUTHORIZED and SERVICE_UNAVAILABLE.

Removed or deferred settings

No one-minute cancellation timer, financial limits, Amazon stock gate, retailer allowlist or dry-run default is part of the running MVP 1 configuration. The trusted BFMR hostname list is a security control, not a retailer allowlist. Test fakes are dependency-injected in automated tests; the testing deployment mode still reserves real inventory.

Security and Ubuntu operations

Boot and desktop session

Run under a dedicated non-root account using systemd with automatic boot start, restart-on-failure and backoff. Depend on local PostgreSQL/network readiness; expected outages should retry within the live process. Use a dedicated persistent Chromium profile restricted to the service owner. Pin Playwright and install its matching browser from the actual application repository.

Headed Chromium requires a display session, not a physical monitor. Provide a secured desktop/remote desktop so Matt can complete login/MFA. A virtual display alone is not an operator interface. Missing session access means a pause, not invisible unattended login. Do not silently enable insecure autologin or expose debugging ports. Validate startup after reboot and session recovery on the Ubuntu host.

Credentials and external content

Scope Gmail to required read/watch access and Pub/Sub to the configured subscription. Keep OAuth, BFMR keys, Telegram token, database credentials and browser sessions outside Git, artifacts, logs and AI inputs. Restrict local secret files. Do not send BFMR API headers into browser automation. Configure API base URLs explicitly; never derive them from an email.

Verify trusted senders and permitted navigation destinations. Do not log token-bearing URLs, authorization headers, cookies, full email bodies, raw account pages or reusable credentials. Stop for access challenges; no stealth, evasion or MFA bypass. Do not expose public webhook, database or remote-desktop ports.

Retention, backups and restore

Retain ordinary logs and intake evidence for 180 days. Keep unresolved attempt/review evidence until disposed and minimal email/command dedupe tombstones beyond payload expiry. Keep selected offer/reservation identity for audit and future purchase matching; define longer archival policy before expansion.

Take encrypted daily PostgreSQL backups off-host and test restoration. Keep secret recovery material separately. Record snapshot/config/schema versions. A stale backup may omit actual submissions: reconcile the account before scheduling and hold affected work when recent actions cannot be reconstructed. A lost Gmail cursor is an incident, not permission to reset and discard missed mail.

Operational failures

Database failure blocks submission and ACK of unpersisted messages. Attempt sanitized fallback logs and alerts where possible. Preserve known Telegram failure and history-gap incidents across restart. Configure power recovery and monitor disk space/watch renewal. Development remains separate from the production appliance, using reviewed releases and restricted access to production secrets.

Acceptance tests for intake and scope

These are required future implementation evidence, not tests executed while writing this specification. Use sanitized fixtures, fake clocks and injected adapter failures; assert durable state and external invocation counts.

Test and trace	Required assertion
AT-01 | FR-001, DATA-001	Valid Pub/Sub delivery is durably stored before ACK; duplicate delivery creates one AUTO command. Database failure prevents ACK.
AT-02 | FR-001, FR-002	Direct BFMR mail works in both modes; approved forwards only in testing. Reject forged/other senders. Both subject layouts work without a retailer channel. Repeated same-deal links collapse; multi-deal and additional-slot mail skip.
AT-03 | FR-003, FR-005	Non-Amazon deals and bundles qualify. Submit quantity 1 for one offer; never one per component/color. No financial, payout, retailer-stock or retailer-permission gate is invoked.
AT-04 | FR-004, FR-011	Any reserved row for the same deal blocks all colors, including manual reservations. After manual release a different email may proceed; duplicate delivery of the completed email does not.
AT-05 | FR-006	Sort discovered work by original receipt time/ID. Complete failed status reads before advancing unrelated queue work. Browser operations never overlap.
AT-06 | FR-007	Persist cursor, stop process, receive new email, restart and catch up without waiting for another notification. Admit downtime email despite old receipt timestamp. Keep admitted backlog; dedupe previously completed work.
AT-07 | FR-007, CON-001	Reconnect continues the saved cursor once. No periodic Gmail polling or Inbox sweep. Expired cursor raises persistent gap and is not silently replaced, even after restart.
AT-10 | FR-010, FR-011	Review resolution submits nothing. Explicit replay is idempotent and current-admission-run only; reject unresolved review/active work. Eligible replay has fresh two-attempt budget.
AT-13 | FR-001, OPS-002	Mode switch rejects testing-forward backlog in production; direct-mail backlog remains eligible. Both configured modes create real reservations when using real adapters.
AT-22 | INT-002, SEC-002	Navigate a trusted BFMR email link without Active Deals lookup; establish stable deal identity. Reject malicious host/lookalike URLs, unapproved redirects and ambiguous forms. Unknown mapping cannot submit.

Acceptance tests for side effects

Test and trace	Required assertion
AT-08 | FR-008	Submit timeout triggers bounded reconciliation with zero automatic resubmits while UNKNOWN. Unresolved result creates review/alert; unrelated work continues after classification.
AT-09 | FR-009	Proved non-creation allows one retry only. Second failure exhausts/alerts. Closed/no-slots skip without retry/alert. No ordinal 3.
AT-11 | OPS-003	Notification failure halts new work but preserves intake. getMe/reachability do not reopen the gate. Required accepted sends do; no startup test or success alert.
AT-12 | CON-001, SEC-001	No direct reservation POST/fetch fallback. No cancellation path. Secrets are redacted. A second instance cannot own the browser.
AT-14 | FR-008, DATA-002	Crash after preparation before click, after click, after response before commit and during reconciliation. Recover using evidence; never blind-resubmit.
AT-15 | FR-009, OPS-001	Restart in RETRY_PENDING preserves ordinal 2 only. Terminal EXHAUSTED is not re-enqueued. New-run qualifying email obeys recovered evidence and live checks.
AT-16 | FR-008, FR-010	Prior uncertainty recreates current-run review. Acknowledgement alone cannot erase uncertainty. Confirmed disposition records evidence without enqueue.
AT-17 | FR-005, FR-008	Wrong quantity, wrong offer/color, possible simultaneous manual action and ambiguous bundle quantity produce CONFLICT/REVIEW. No automatic cancellation or correction.
AT-18 | OPS-003	Restart preserves Telegram block and outbox identity. Lost acceptance response can duplicate an alert but never creates another BFMR submission.
AT-19 | DATA-001, SEC-001	Database/lock loss stops new actions. Supervisor prevents old/new browsers overlapping. Stale backup restore requires reconciliation.
AT-20 | FR-004, INT-001	Verify tracker auth, account alignment, date range/buffer, all pages, IDs/status/quantity/offer mapping and immediate visibility. Incomplete reads never prove absence.
AT-21 | OPS-001, OPS-002	Ubuntu boot starts pinned headed Chromium with secured visible session. Login/MFA and restart recovery work without a physical monitor.
AT-23 | FR-012	Successful reservation remains untouched by application timers/restarts. No purchase or cancellation request. Matt releases manually after designated live tests.

Implementation plan and evidence gates

Build sequence

1. Audit actual code and repository revision; do not infer implementation status from contradictory handoff sections. Identify reuse, remove incompatible reservation request paths and capture sanitized fixtures. Pin supported runtime/library versions.

2. Implement PostgreSQL schemas, singleton ownership, durable cursor/queue, command state machine and incident outbox. Prove transaction and crash invariants with fake adapters.

3. Implement Pub/Sub pull and serialized Gmail history continuation. Remove per-restart baseline resets and receipt-time filters. Prove downtime catch-up, first initialization, watch renewal and persistent history-gap behavior.

4. Implement direct authenticated BFMR navigation, stable deal/offer identity and complete My Tracker read contract. Cover bundles and arbitrary retailers; remove business eligibility filters.

5. Implement quantity-one website submission, reconciliation, bounded retry, Telegram gates and CLI. No production write fallback or cancellation path. Complete offline failure tests before designated real reservations.

6. Package supervised Ubuntu service, secure setup, migrations, backup/restore, runbook and acceptance evidence. Release only after the gates below are satisfied.

Gate	Evidence required
G-001 BFMR contract	Sanitized browser/API fixtures prove account/deal identity, direct link resolution, form/offer mapping, bundle quantity, tracker date filters/paging, RID, release removal and immediate success visibility.
G-002 Email contract	Real sanitized MIME/auth evidence for direct and forwarded messages, both subject families, duplicate links, single bundle and multiple-deal/slot notices.
G-003 Ubuntu runtime	Actual repository/revision; pinned Chromium; secured desktop; boot, login/MFA and single-owner recovery evidence.
G-004 Catch-up boundary	Automated restart/reconnect/cursor tests; no receipt cutoff, no polling/sweep. Document expired-history limitation and obtain a separate decision if lossless long-outage recovery is required.
G-005 End-to-end	Designated real test reservation, duplicate/manual reservation check, manual release, Telegram failure/recovery, crashes and backup restore. Record versions, times, fixtures and outcomes.

Do not mark gates complete from narrative claims alone. Deliver repository location/revision, migrations, dependency lockfile, non-secret configuration example, sanitized fixtures, adapter results, operator runbook and acceptance report with this specification.

Deferred financial decisions and history

None of this page is executable MVP 1 scope. The user ultimately removed all eligibility evaluation from MVP 1. The earlier decisions are retained for traceability and possible later-MVP requirements work; no later milestone assignment or activation is implied.

Earlier clarification	Disposition after the final scope change
Reserve then evaluate	Initially approved: reserve 1, confirm creation, evaluate eligibility, release failed deals. Withdrawn for MVP 1 when all validation was removed.
Automatic failed-deal release	Initially approved through BFMR cancellation API. Superseded for MVP 1 by all releases manual. API cancellation may be revisited only in later scope.
Financial criteria	Four criteria discussed: exposure limit, total cost no greater than payout, minimum cost and maximum cost. All excluded from MVP 1.
Inclusive boundaries	Earlier accepted: exposure may equal its limit; unit cost may equal either minimum or maximum; total cost may equal payout.
Exposure source and formula	Earlier accepted: existing unpaid exposure from BFMR My Tracker, measured as expected payouts; add the proposed Amazon item cost, not its expected payout. The deliberate mixed basis was confirmed.
Other unpurchased reservations	Whether they count toward exposure was not answered. The user instead removed all validation. Remains unresolved if financial scope returns.
Extra account restriction	Rejected the handoff rule blocking a retailer account with any unpaid transaction. Earlier financial discussion allowed multiple unpaid transactions within exposure limit.
Extra retail-price restriction	Rejected blocking solely because Amazon price exceeds BFMR stated retail price. Earlier criterion was total cost no greater than payout, with other agreed limits.
Checkout and extra charges	Rejected entering checkout to obtain final total. Zero shipping, tax and other charges was a temporary MVP 1 assumption, explicitly expected to change later. Now inactive; do not automatically carry zero charges into a later purchasing MVP.
Amazon stock and bundles	Initially included Amazon in-stock and non-bundle criteria. Superseded: no stock check, all retailers and bundles permitted.
Eligible reservation and purchase	Earlier agreement left eligible reservations for manual purchase. Current MVP 1 simply leaves successful reservations untouched; no eligibility result exists.

Original roadmap: MVP 2 evaluation, MVP 3 purchasing/order update, MVP 4 shipment tracking, MVP 5 extra slots, MVP 6 multiple-deal mail. This review changes MVP 1 only; reconcile later milestones before implementing them. Prior Amazon gift requirements remain later purchasing context, not an MVP 1 browser action.

Source defects and evidence register

Finding	Required treatment
Handoff implementation status	September 26 update says reservation MVP exists, while older summary/status rows say no reservation side effects. Treat earlier claims as stale under its supersession note; verify actual code before claiming current status.
API response examples	Active Deals example depicts an object and string-only fields; handoff reports deals array, nullable closing_at/deal_information and flexible items. Not an MVP 1 discovery dependency. Preserve verified shapes if later used.
My Tracker schema gaps	Published sample is my_tracker: [{}]. Fields, pagination, date behavior, RID and immediate visibility are not proven by it. Validate before duplicate detection or retry relies on them.
API cancellation examples	Declared form encoding disagrees with JSON examples, whose quoting is also invalid; reservation cancellation uses an internal HTTP hostname. Outside MVP 1. Do not guess a production write contract from the sample.
Insurance city inconsistency	Field description allows city to be blank, but sample comment says omitted city can be required. Outside MVP 1; resolve against authoritative provider behavior if later used.
Tracker credential omission	V1.4 configuration omitted explicit BFMR read credentials while requiring My Tracker API. V1.5 requires key/secret references for tracker reads; no reservation-creation credential/endpoint is invented.
Historic live evidence	Reported successful redirect, tracker immediacy/purge and earlier tests are attributed observations, not newly validated facts. Reproduce target-environment contracts through G-001 to G-005.

Source precedence

S1: Final explicit user answers in this conversation, 9 October 2026. **S2:** BFMR_Solution_Design_Specification_v1.4.pdf, 9 October 2026, retained where not superseded. **S3:** BFMR_Automation_Project_Handoff_Updated_20260926.docx, historical implementation and design evidence. **S4:** BFMR_API.txt, supplied provider reference with known defects. **S5:** mvp.txt, original roadmap. **S6:** bfmr_deal_criteria.txt, deferred financial rules.

External implementation references

Gmail push and history: developers.google.com/workspace/gmail/api/guides/push and /guides/sync. Pub/Sub: docs.cloud.google.com/pubsub/docs/subscribe-best-practices. Telegram: core.telegram.org/bots/api. Playwright: playwright.dev/docs/api/class-page, /docs/browsers and /docs/ci. These are implementation references; provider behavior and versions require verification at implementation. No live account validation was performed to produce v1.5.

Traceability and supersession checklist

Requirement	Design pages	Acceptance evidence
FR-001, FR-002	6-7	AT-01, AT-02, AT-13
FR-003, FR-005	7-9	AT-03, AT-17
FR-004, INT-001	8-9	AT-04, AT-20
FR-006	9, 12-13	AT-05
FR-007, CON-001 intake	5-6, 10, 12	AT-06, AT-07
FR-008, FR-009	8-10, 13	AT-08, AT-09, AT-14 to AT-17
FR-010, FR-011	9-10, 14	AT-04, AT-10
FR-012	2-3, 9, 14	AT-12, AT-23
OPS-001, OPS-002	10, 15	AT-13 to AT-16, AT-19, AT-21
OPS-003	11, 14	AT-11, AT-18
INT-002, SEC-002	5, 7, 15	AT-02, AT-22
SEC-001, DATA-001, DATA-002	12-15	AT-01, AT-12, AT-14, AT-19
CON-001 UI submission	5, 8-9	AT-12, G-001

Remove these superseded MVP 1 behaviors

Windows/Edge deployment assumptions; Active Deals lookup as mandatory link resolution; Amazon-only filtering; retailer stock checks; bundle rejection; financial evaluation; one-open-transaction account gate; retail-price ceiling; timer-based or failed-eligibility cancellation; cancellation API writes; dry-run deployment default; required retailer subject channel; restart Gmail cursor reset and receipt-time exclusion.

Retain these controls

One unit and one worker; trusted email intake; correct account/deal identity; My Tracker duplicate and confirmation reads; durable dedupe/cursor/attempts; safe uncertain-result reconciliation; bounded retry; manual review; Telegram failure gating; secure headed Chromium; manual release; no purchasing; no multiple-deal or additional-slot processing.

Documentation completion versus system completion

V1.5 is the consolidated design baseline. It does not certify source-code behavior, working credentials, provider contracts, successful tests or production readiness. An implementation agent must satisfy the evidence gates and update any engineering decisions that change, with corresponding requirements and acceptance tests. Source documents remain unchanged historical references.